Planetary Rover — Solar Energy and Charging Dynamics

Assignment D1-V7 | AI4Ro2 | **Hani Bouhraoua** (8314923) | Planner: ENHSP (`-planner sat-hadd`)

Domain & Problem

A single solar-powered rover explores a 4-node **directional** graph (A,B,C,D) and must **visit every location without depleting its battery**. Moving consumes energy; sun-exposed zones (B,C) recharge it, with efficiency varying by location. The model uses only two object types (`rover`, `location`); everything else is a **predicate** (`at`, `connected`, `solar-zone`, `visited`, `daytime`) or a **numeric fluent** (`battery-level`, `max-battery`, `move-cost`, `charge-rate`, `time-of-day`). **Q1** is classical PDDL with numeric fluents; **Q2** is PDDL+ with processes and events.

Modelling Choices

- **Energy as a constraint.** `move` requires `(>= battery-level move-cost)` and decreases the battery, so the rover cannot cross an edge it cannot afford. `move-cost` (consumption) is kept separate from `charge-rate` (generation), and `charge-rate` is per-location to capture varying efficiency.
- **Q1 recharge = action.** Instantaneous `(increase battery charge-rate)` at a solar zone. Justified because classical PDDL has no time; Q2 makes it continuous.
- **Q2 recharge = process.** `:process charging` raises battery by `(* #t (charge-rate ?l))` — continuous over time — only while the rover is *at* a solar zone, in `(daytime)`, and not full. The rover does not *choose* to charge; the world does it automatically.
- **Q2 nightfall = event.** `:event nightfall` fires when `(>= time-of-day 100)` and flips `(daytime)` off. The `charging` process and the event are not linked explicitly — they coordinate through the shared `(daytime)` fact, so charging stops the instant night falls. A `time-flow` process advances the clock.
- **Charge-rate rescaling.** Q1 rates (15, 25) are one-shot jumps; Q2 rates (0.2, 0.5) are per-tick, since they multiply by elapsed time.

Plan Walkthrough (behaves as expected)

Q1-A (battery 50): `move` A→B→C→D, no recharge — optional. **Q1-B** (battery 25): inserts `recharge` at C — forced, else stranded at C ($5 < 10$).

Q2 varies only `time-of-day` to show timing drives feasibility: *easy* (t=0): 3 moves, no charging. *tight* (t=0): a valid 47-tick plan that charges across both zones — valid but not time-optimal (`hadd` counts actions, not time). *fastzone-only* (t=79): only 21 ticks of daylight force the *optimal* 20-tick plan (charge at C only). *infeasible* (t=85): 15 ticks of daylight $<$ 20 needed $\Rightarrow$ **unsolvable** — nightfall stops charging too soon. All outputs match the hand-traced predictions and are stored in `codes/outputs/`.

Limitations & Known Issues

- **Optimality not guaranteed.** Satisficing search returns valid but sometimes time-wasteful plans (the 47-tick *tight* case); a `(:metric)` or a tighter deadline is needed to force optimality.
- **Instantaneous moves** in Q1/Q2. An optional extension makes travel take time, modelled as `start-move` action + `travel` process + `arrive` event (ENHSP does not support `:durative-action`); it shifts every feasibility boundary earlier without changing the lesson.
- **Edges assumed traversable** (no obstacles / task–motion gap); **single agent**; **deterministic** (no sensor/energy uncertainty — would need MDP/POMDP, unsupported by PDDL planners).

Sample Output (Q2 tight)

```
0:  (move rover1 locA locB)
0:  ---waiting-- [45.0]    ; charging at B (rate 0.2)
45.0:  (move rover1 locB locC)
45.0:  ---waiting-- [47.0]  ; charging at C (rate 0.5)
47.0:  (move rover1 locC locD)    Elapsed Time:  47.0
```